

APUNTES Y EJERCICIOS

FULL STACK DEVELOPER

1ED - 2025

HTML + CSS + JS

Alumna: Maria Luz Opazo



CILSA

ong por la inclusión

profesores: Jesús Chirinos y David Huertas



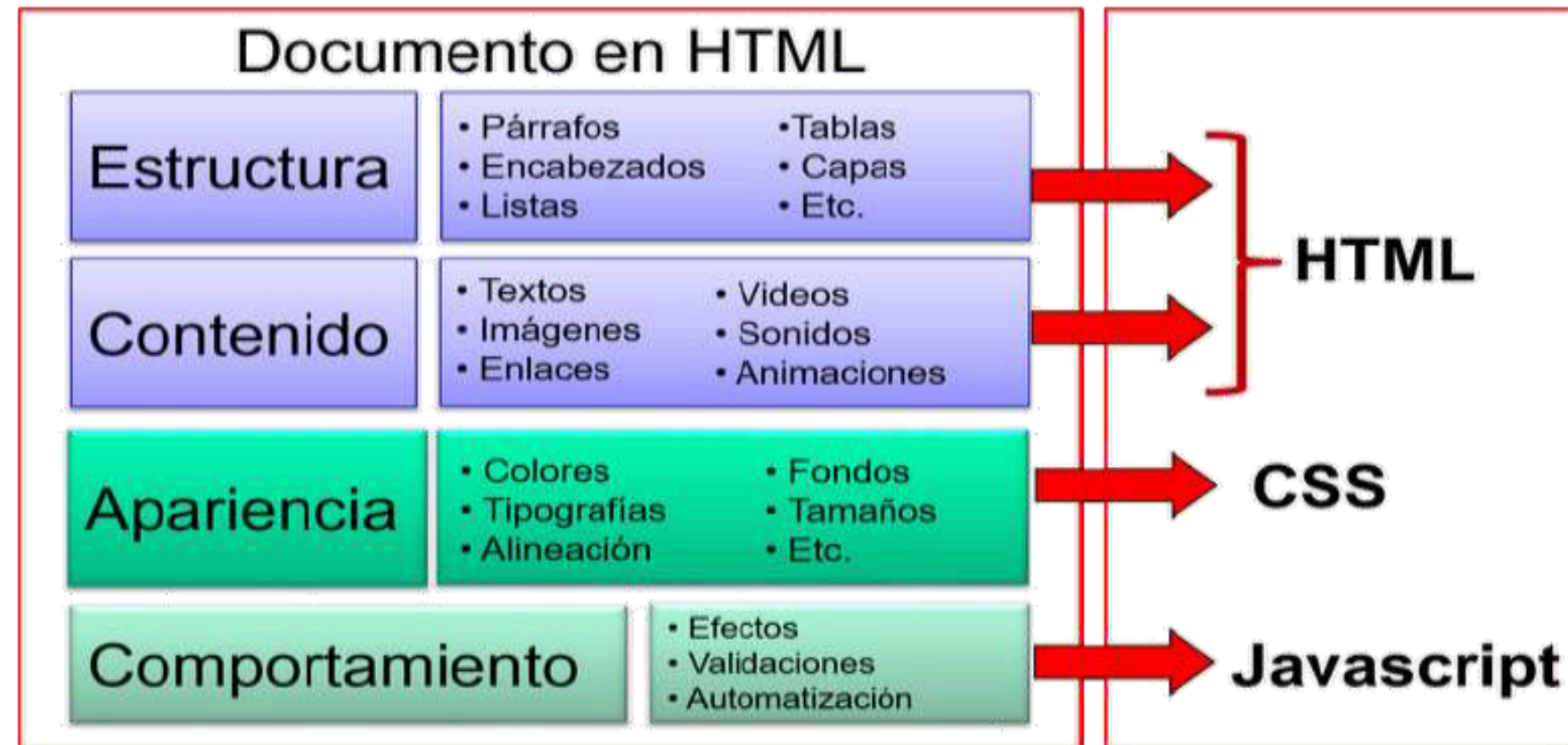
HTML + CSS

estructura de un documento

Un documento HTML está formado por tres partes:

- Una línea que contiene información sobre la versión de HTML (no siempre),
- Una cabecera (delimitada por el elemento HEAD), y
- Un cuerpo, con el contenido del documento (delimitado por el elemento BODY FRAMESET).

Todo el documento tiene que ir entre las etiquetas <HTML></HTML>



Estructura básica



HTML

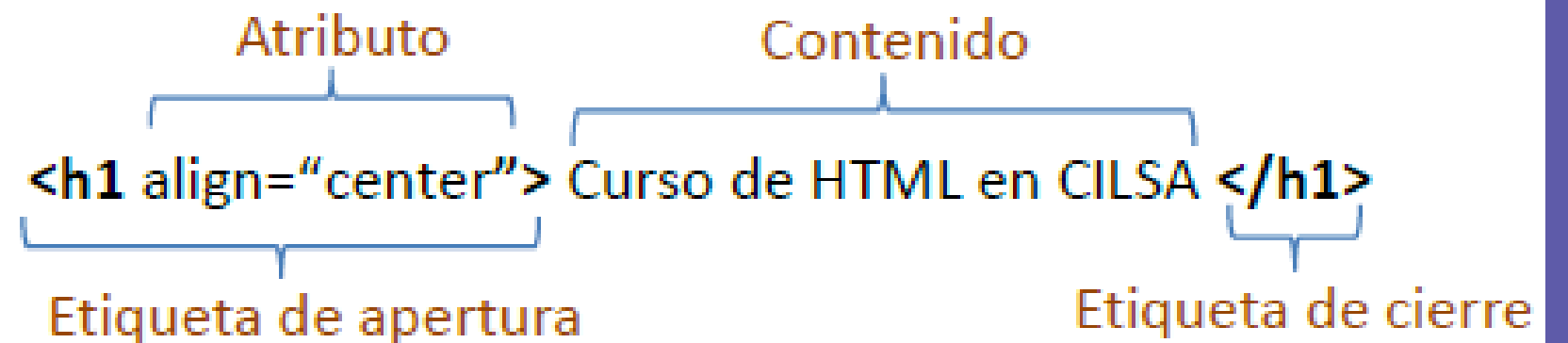
Lenguaje de Marcado de Hipertexto



HTML funciona con **etiquetas** (tags) que nos permiten **organizar** el contenido de una página web



Etiquetas que tienen una apertura y un cierre



- El contenido puede ser texto u otra etiqueta (como hijos).
 - Los atributos proveen información adicional sobre el elemento o etiqueta, a la vez que lo modifican.
- La mayoría de atributos tienen un valor; el valor modifica el atributo. Por ejemplo, que `h1` debería estar alineado hacia el centro

HTML

importante para tener en cuenta
en la estructura de la pagina >>>

Orden de apertura y de cierre

La etiqueta en primero en cerrarse tiene que ser la última que se abrió. Por ejemplo:

`<p>.....<strong>.....</p></strong>` ➡ Incorrecto

`<p>.....<strong>.....</strong></p>` ➡ Correcto

- **Doctype**-> “tipo de documento” indica al navegador que el código html en el que está escrita la página es en la versión 5.
- **Html**-> elemento principal, “el papá de todo”. No hay elemento por encima, dentro hay dos elementos internos, head y body
- **Head**-> cabecera, contiene todos los elementos no visibles que ayudan a que la página funcione correctamente, se incluyen etiquetas que ofrecen información principalmente (metadatos) a los navegadores, buscadores y otras páginas.
- **Body**-> están incluidos todos los elementos visuales-estructurales, como encabezados, párrafos, listas, imágenes e hipervínculos. O sea, todo el contenido que puede ver el usuario en el navegador.

Atributos

- **lang=es** => es para especificar en qué idioma está escrita la página, por ejemplo para los buscadores.
- **Charset="utf-8"** => indica al navegador que tipo de caracteres contiene la página. El valor utf-8 podremos crear en la mayoría de los sistemas de escritura español, inglés, francés, etc.

- forma de ordenar la apertura y el cierre de una etiqueta

HTML

diccionario de etiquetas y su función

`<!DOCTYPE>` Define el tipo de documento

`<html>` define el inicio del documento HTML

`<script>` incrusta un script en una web

`<head>` define la cabecera del documento HTML

`<title>` define el título de la página

`<link>` para vincular el sitio a hojas de estilo o iconos

`<style>` para colocar el estilo interno de la página; ya sea usando CSS u otros lenguajes similares

`<body>` define el contenido principal o cuerpo del documento

`<header>` División estructural en la parte del contenido

`<article>` Representa una composición auto-contenida en un documento

`<h1>` a `<h6>` encabezados o títulos del documento

`<table>` define una tabla

`<tr>` fila de una tabla

`<td>` celda de una tabla (debe estar dentro de una fila)

`<footer>` representa el pie de un documento

`<a>` hipervínculo o enlace

`<div>` división de la página

`<img>` imagen

```<ol>``` etiquetas para listas

`<b>` texto en negrita (`strong` también)

`<i>` texto en cursiva

HTML

diccionario de etiquetas y su función

`<s>` texto tachado

`<span>` Sirve para diferenciar un texto de otro

`<br>` Sirve para provocar un salto de línea

`<aside>` Define el contenido lateral del contenedor de una página

`<!--...-->` Define un comentario

`<address>` Define la información de contacto del autor

`<audio>` Define contenido de sonido

`<button>` Define un botón clickeable

`<em>` Define énfasis en un texto

`<figure>` Especifica auto-contenido

`<input>` Define un control de entrada de texto

`<label>` Define el rótulo para un elemento `<input>`

`<mark>` Define texto resaltado o marcado

`<nav>` Define un link de navegación

`<section>` Define una sección de un documento

`<small>` Define texto pequeño

`<span>` Define una pequeña sección de un documento

`<strong>` Define un texto en negrita

`<textarea>` Define un control de entrada de múltiples líneas

`<video>` Define un vídeo o película

La mayoría de etiquetas deben cerrarse como se abren, pero con una barra (`</>`) tal como se muestra en los siguientes ejemplos:

`<table><tr><td>Contenido de una celda</td></tr></table>.`

CSS

lenguaje de hojas de estilo en cascada

- se usa para **estilizar** elementos escritos en un lenguaje de marcado como HTML

estructura de css

selector declaración declaración

↓

p { **color: blue;** **font-family: Avenir;** }

↑ ↑ ↑ ↑

propiedad valor propiedad valor

Veamos un ejemplo:

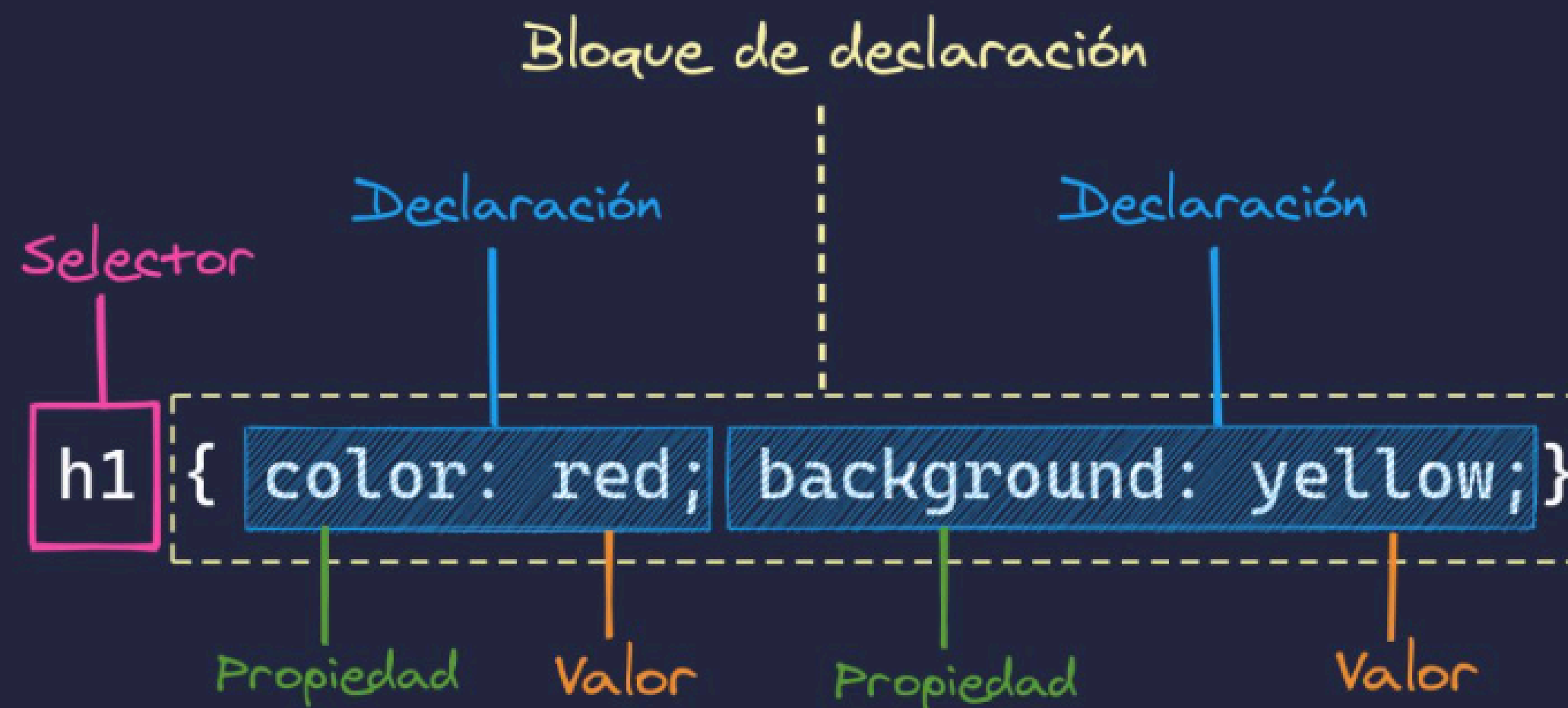
Todos los elementos **<p>** aparecerán en color azul y en negrita.

```
1. <style>
2. p {
3.   color: blue;
4.   text-weight: bold;
5. }
6. </style>
```

CSS

lenguaje de hojas
de estilo en cascada

Anatomía de un regla de estilo



Selector

Es un descriptor que le permite dirigirse a elementos específicos de la página.

Propiedades

Son los atributos para los que se pueden especificar valores.

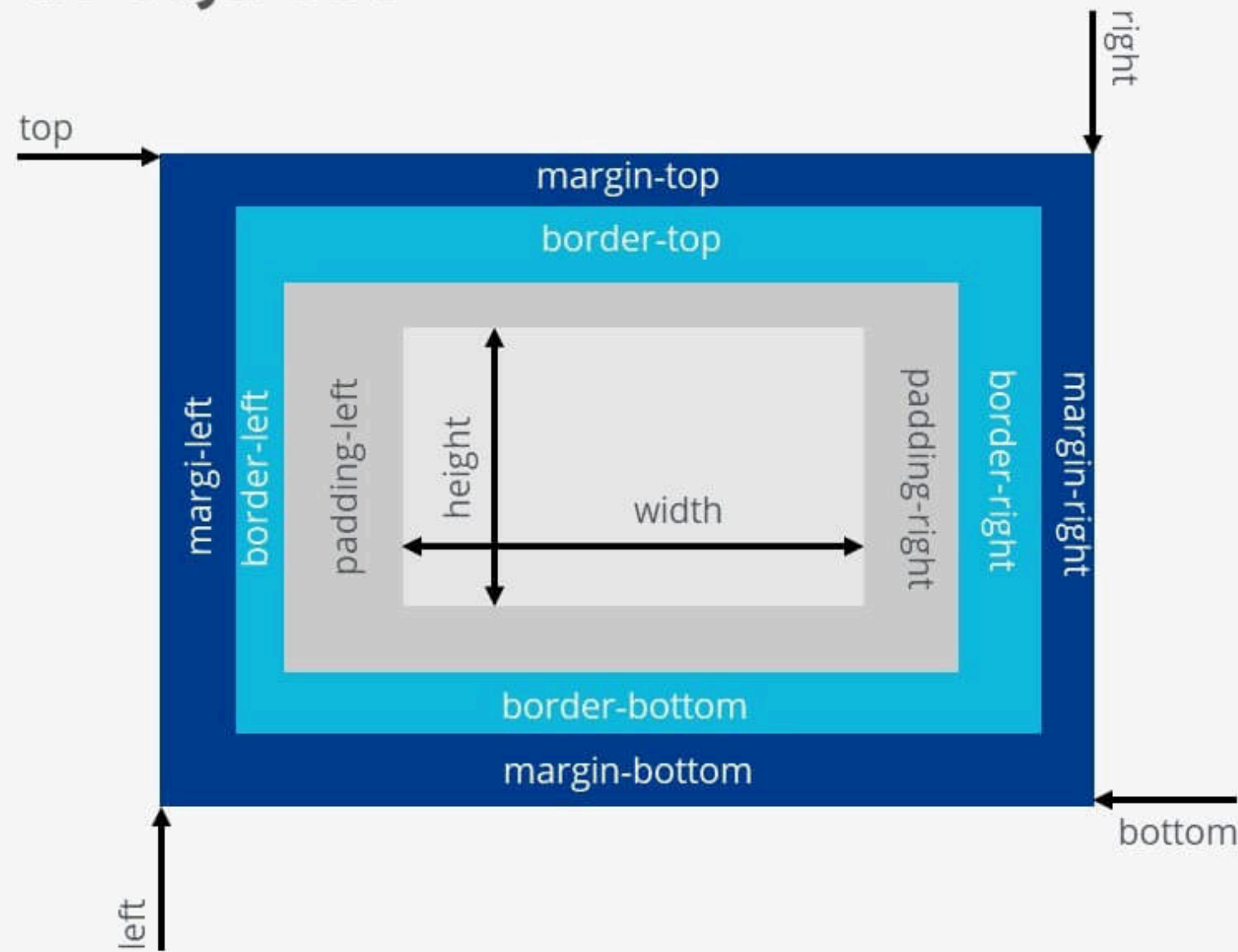
Declaración

Es una combinación de una propiedad y un valor.

CSS

lenguaje de hojas
de estilo en cascada

Modelo de caja CSS



- `<html lang="ES">`
- `<meta name="descripcion" content="esta es mi primer web" />`
- `<article id="nombre"> ESTE ES UN ARTICULO </article>`
- `id = "texto"` (Establece un identificador único a cada elemento dentro de una página HTML)
- `class = "texto"` (Establece la clase CSS que se aplica a los estilos del elemento)
- `style = "texto"` (Establece de forma directa los estilos CSS de un elemento)
- `Font-size: 20px;`
- `Font-family: sans-serif;` (elegir la fuente a usa)
- `Font-weight: lighter;` (la fuente queda más fina también usar el "bold")
- `Font: bold 14px sans-serif;`
- `Border: 1px solid black;`
- `Border-radius:`
- `Text-align: right; left; center;`
- `Width: 960px;`
- `Margin: auto; Margin-left: auto; Margin-right: auto;`
- `Margin-top: 200px; Margin-top Width`
- `Padding: 40 px;`
- `List-style: none;`
- `Display: block; Display: inline-block;`
- `Módulo CSS:`
- `Float: right; left; center;`
- `Clear: both;` (debe ir en el pie de pag para que los elementos que floten entren)

JS JavaScript

- Los programas escritos con JavaScript se pueden probar directamente en cualquier navegador sin necesidad de procesos intermedios

este programa permite:

- Crear **efectos especiales**
- Se pueden **validar datos** de un formulario.
- Crear **rollovers** (cambia la imagen cuando se pasa el mouse).
- Crear **navegadores desplegados**.
- Se puede realizar la **apertura de ventanas secundarias**.
- Trabajar con la **barra de estado**, Etc.

```
<!DOCTYPE html PUBLIC>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>El primer script</title>
    <script type="text/javascript">
      alert("Hola Mundo!");
    </script>
  </head>
  <body>
    <p>Esta página contiene el primer script</p>
  </body>
</html>
```

¿Dónde se escribe un JAVASCRIPT?

La integración de JavaScript y HTML es muy flexible, ya que existen al menos tres formas para incluir código JavaScript en las páginas web.

- La programación de JavaScript se realiza en un documento HTML
- Se pueden incorporar en la cabecera entre las etiquetas <HEAD> </HEAD>
- En el cuerpo de la página entre las etiquetas <BODY> </BODY>, a esto se le llama código incrustado.
- Se pueden introducir varios scripts en una misma página con etiquetas <script> diferentes
- O se puede realizar un archivo externo.
- Cuando existen varias funciones probadas y que pueden servir a mas de un programa se puede realizar un archivo externo.
- El archivo externo normalmente tiene la extensión .js y se lo incluye de la siguiente manera, suponiendo que el archivo con las funciones se llame externo:

```
<script src="externo.JS"> </script>
```



variable



- **Normas para nombres de variables:** Deben ser descriptivos y reflejar el valor que guardan.
- Pueden **contener** letras, números, guiones bajos (_) y el signo de dólar (\$).
- **No pueden** empezar con un número.
- Son sensibles a **mayúsculas y minúsculas** (miVariable no es lo mismo que mivariable).
- Hay **palabras reservadas** que no se pueden usar como nombres de variables (ej: if, for, function).
- **Declaración de variables:** Antes de usar una variable, debes declararla. Se hace con las palabras clave let, const o var.
- **let:** Se utiliza para variables que pueden cambiar su valor.
- **const:** Se utiliza para variables cuyo valor no debe cambiar una vez asignado.
- **var:** Es una forma más antigua de declarar variables y tiene algunas diferencias de ámbito con let y const. Es preferible usar let y const en el código moderno.
- **Ámbito de las variables:** Define dónde una variable es "visible" y se puede utilizar en tu código.
- **Ámbito global:** Declaradas fuera de cualquier función, son accesibles desde cualquier parte del código.
- **Ámbito local** (de función): Declaradas dentro de una función, solo son accesibles dentro de esa función (con var).
- **Ámbito de bloque:** Declaradas dentro de un bloque de código (como un if o un for) con let o const, solo son accesibles dentro de ese bloque.

Operadores lógicos y relacionales

Operadores lógicos: Funcionan con valores booleanos y devuelven los valores “verdadero”, “falso” y “negación”.

Operador	Significado
&&	AND
	OR
!	NOT

Operadores relacionales: Sirven para comparar valores.

Operador	Significado	Operador	Significado
<	Menor a	<=	Menor o igual a
>	Mayor a	>=	Mayor o igual a
==	Igual a	!=	Diferente a

arrays en javascript

los arrays son colecciones de variables que se cuentan desde cero 0

operadores logicos y relacionales

```
<script>
    var granPoblacion = ["Madrid", "Barcelona", "Valencia", "Sevilla"];           //declaramos el
array
    document.write("granPoblacion[2]: " + granPoblacion[2] + "<br>");           //imprimimos el
valor '2'
    granPoblacion[2] = "Málaga";                                               // cambiamos el
valor '2'
    document.write("Cambiamos Valencia por: " + granPoblacion[2] + "<br>");     //imprimimos el
nuevo valor '2'
    granPoblacion[4] = "Zaragoza";                                             //añadimos el valor Za
ragoza al array
    document.write("Hemos añadido al array: " + granPoblacion[4]);           //imprimimos el
nuevo valor
</script>
```

Formulario

actividad 1 :

Consigna:

- Tarea 1: crear un formulario de datos personales con los campos; Nombre, Apellido, Email, Fecha de nacimiento, País de residencia, un botón de enviar y 1 o 2 botones más para cambio de estilo, sin recursos, solo HTML y CSS

Formulario de Datos Personales

Nombre:

Apellido:

Email:

Fecha de Nacimiento:

País de Residencia:

Enviar

Estilo Claro

Estilo Oscuro

Formulario

actividad 2 :

Consigna:

- Tarea 2: crear un formulario de datos personales con los campos; Nombre, Apellido, Email, Fecha de nacimiento, País de residencia, un botón de enviar y 1 o 2 botones más para cambio de estilo, sin recursos, solo HTML y CSS. Modificando la tarea 1, pero ahora con un estilo de alto contraste.

Formulario de Datos Personales

Nombre:

Apellido:

Email:

Fecha de Nacimiento:

País de Residencia:

Enviar

Alto Contraste
Azul

Alto Contraste
Amarillo

Formulario de Datos Personales

Nombre:

Apellido:

Email:

Fecha de Nacimiento:

País de Residencia:

Enviar

Alto Contraste
Azul

Alto Contraste
Amarillo

Formulario de Datos Personales

Nombre:

Apellido:

Email:

Fecha de Nacimiento:

País de Residencia:

Enviar

Alto Contraste
Azul

Alto Contraste
Amarillo

Formulario

actividad 3 :

- Tarea 3: Tomar el ejercicio 1 y 2 y agregar las diferentes validaciones en HTML necesarias

Formulario de Datos Personales

Nombre:

Apellido:

 Utiliza un formato que coincida con el solicitado

Email:

Fecha de Nacimiento:

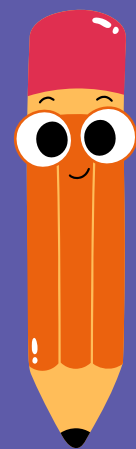
País de Residencia:

EnviarEstilo ClaroEstilo Oscuro

validaciones:

- se implementan para asegurar que los datos ingresados por el usuario cumplan con ciertos criterios o reglas antes de ser enviados al servidor para su procesamiento. Su objetivo principal es garantizar la calidad, integridad y seguridad de la información que se recibe.

las validaciones:



- previenen errores
- Mejoran la calidad de los datos
- Refuerzan la seguridad
- Guían al usuario

`required` : Indica que un campo debe ser llenado antes de enviar el formulario.

`type="email"` : Asegura que el valor ingresado tenga un formato de correo electrónico válido.

`type="date"` : Muestra un selector de fechas y asegura que el valor sea una fecha válida.

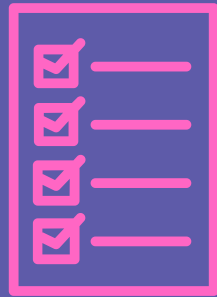
`minlength="n"` y `maxlength="n"` : Definen la longitud mínima y máxima de caracteres permitidos en un campo de texto.

`min="n"` y `max="n"` : Definen los valores mínimo y máximo permitidos para campos numéricos o de fecha.

`pattern="regex"` : Permite definir una expresión regular (regex) que el valor del campo debe cumplir. Es muy potente para validar formatos específicos.

`step="n"` : Especifica los intervalos válidos para un valor numérico.

`placeholder="texto"` : Aunque no es una validación en sí, proporciona una pista visual al usuario sobre qué tipo de información debe ingresar.



TESTING

objetivos de aprender sobre testing:

- aprender sobre el manejo de errores
- aprender sobre los diferentes niveles de prueba

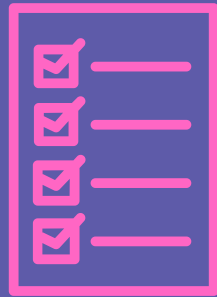
Tester / probador: Asegura la calidad del producto mediante pruebas

- **pasos** para el proceso de desarrollo de un software

- **Especificación y análisis de requisitos:** Definir qué debe hacer el software.
- **Diseño:** Planificar la arquitectura y la interfaz del sistema.
- **Codificación:** Escribir el código del programa.
- **Pruebas / Testing:** Verificar el funcionamiento y la calidad del software.
- **Instalación y paso a producción:** Desplegar el software para su uso.
- **Mantenimiento:** Realizar ajustes y correcciones posteriores.

tipos de requisitos a la hora de definir

- **Requisitos funcionales:** Describen lo que el sistema debe hacer.
- **Requisitos no funcionales:** Especifican cómo debe realizar sus funciones (disponibilidad, usabilidad, etc.).



TESTING

objetivos de aprender sobre testing:

- aprender sobre el manejo de errores
- aprender sobre los diferentes niveles de prueba

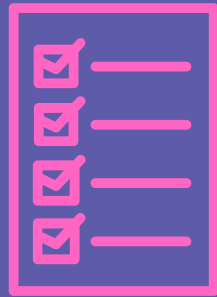
Tester / probador: Asegura la calidad del producto mediante pruebas

fundamentos:

- **Error, defecto y fallo:** Diferenciando la acción humana incorrecta, el desperfecto en el sistema y la manifestación visible del defecto.
- **Requerimiento y calidad:** Definiendo el requerimiento como una necesidad obligatoria y la calidad como el grado de satisfacción de esos requerimientos y las expectativas del cliente.
- **Atributos de la calidad:** Dividiéndolos en funcionales (correctitud y completitud) y no funcionales (fiabilidad, usabilidad, portabilidad, eficiencia, mantenibilidad).

proceso de testing

- **Planificación y control:** Definición del alcance, objetivos, estrategias y monitoreo del progreso.
- **Análisis y diseño:** Revisión de elementos, identificación de condiciones y diseño de casos de prueba.
- **Implementación y ejecución:** Preparación y ejecución de pruebas, registro de resultados e informe de defectos.
- **Evaluación de criterio de salida y reportes:** Comparación de resultados con los criterios definidos y elaboración de informes.
- **Actividades de cierre:** Recolección de información, verificación de entregables y análisis de lecciones aprendidas.



TESTING

objetivos de aprender sobre testing:

- aprender sobre el manejo de errores
- aprender sobre los diferentes niveles de prueba

Tester / probador: Asegura la calidad del producto mediante pruebas

TAREAS DE UN TESTER

- REUNIRSE CON USUARIOS DEL SISTEMA PARA COMPRENDER EL ALCANCE DE LOS PROYECTOS.
- TRABAJAR CON DESARROLLADORES DE SOFTWARE Y EQUIPOS DE SOPORTE.
- LLEVAR A CABO PRUEBAS DE ESTRÉS, PRUEBAS DE RENDIMIENTO, PRUEBAS FUNCIONALES Y PRUEBAS DE ESCALABILIDAD.
- ESCRIBIR Y EJECUTAR SCRIPTS DE PRUEBA.
- REALIZAR PRUEBAS MANUALES Y AUTOMATIZADAS.
- PRUEBAS EN DIFERENTES ENTORNOS, INCLUYENDO WEB Y MÓVIL.
- ESCRIBIR INFORMES DE FALLOS.



TESTING

A partir del formulario elaborado para la tarea 1, se pide:

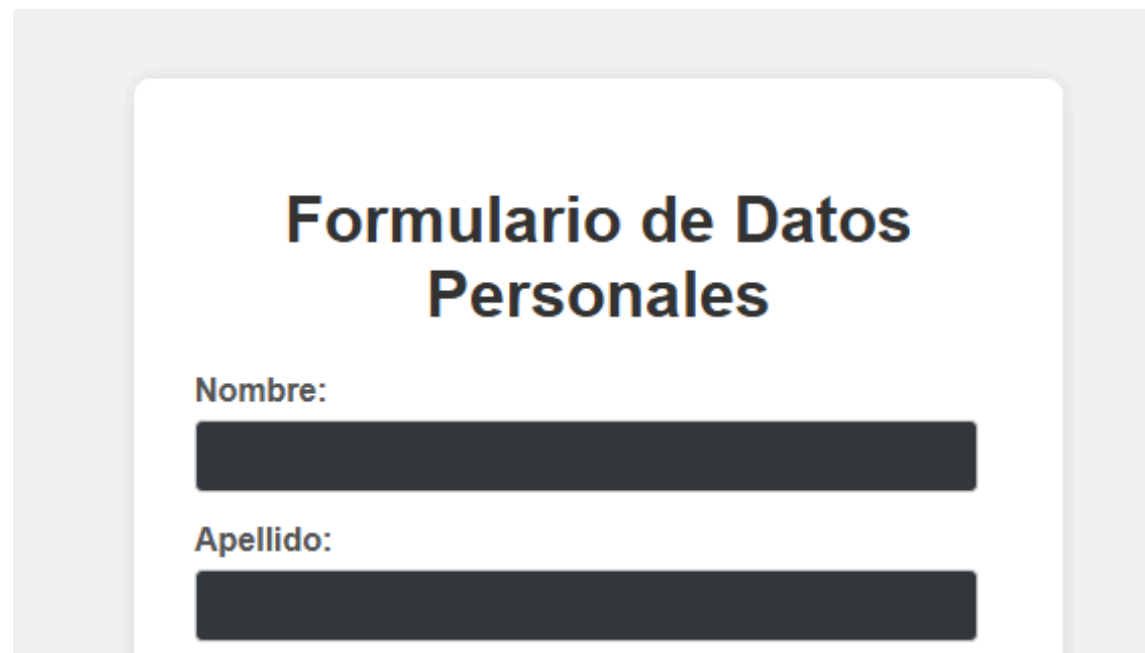
- Detallar los pasos que realizan para poder cargar el mismo.
- Indicar para cada campo cuales son los valores válidos y cuales inválidos. ¿Se realiza esa validación de alguna forma?

Realizar una exploración del formulario y verificar si mediante el lector de pantalla NVDA pueden acceder a cada campo mediante la tecla TAB y completa el mismo. ¿Hay algún campo o acción que no resulte accesible?

PASOS:

1 _ Acceso al formulario:

ejecutar el archivo html del formulario 1



The screenshot shows a web form titled "Formulario de Datos Personales". It contains two input fields: "Nombre:" and "Apellido:". The form is displayed on a light gray background.

2 _ Completar los campos de nombre y apellido:

ingresar un valor en cada campo para comprobar



utilice un nombre de referencia extra largo (tome de referencia el de la película de "gladiador")

Nombre:

Máximo Décimo Meridio Comandante de los Ejércitc

Apellido:

meridio

PASOS:

3 _ completar el campo de email:

prueba 1 con una cuenta de correo

Email:

comandante@hotmail.com

prueba 2 con valores incorrectos (en este caso con números)

Email:

123456

5 _ selecciono una opción en el país de residencia :

prueba con una fecha aleatoria

País de Residencia:

Argentina

4 _ completar el campo de fecha de nacimiento :

prueba 1 con una fecha aleatoria

Fecha de Nacimiento:

30/01/1977

prueba 2 SIN el registro de la fecha

Fecha de Nacimiento:

dd/mm/aaaa

6 _ selecciono el botón enviar:

Enviar

Estilo Claro

Estilo Oscuro

RESULTADOS:

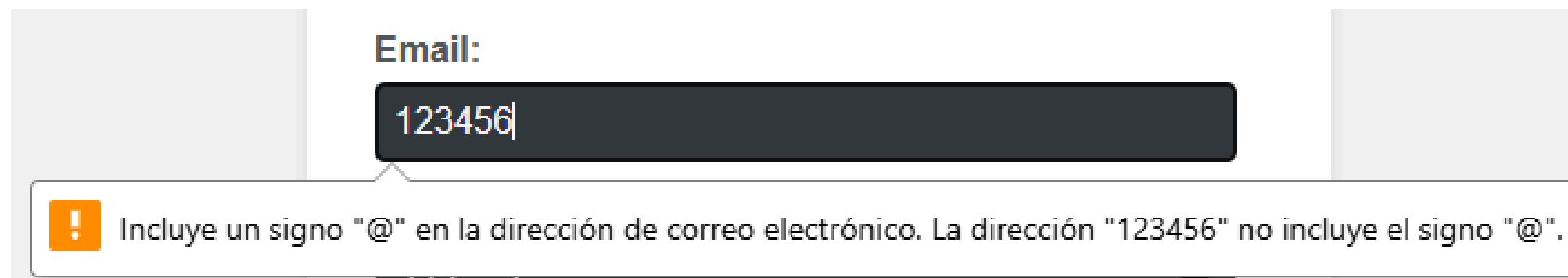
que tipo de validaciones se hicieron al tocar el botón enviar?

en el caso de la prueba 2 , al completar con valores incorrectos el campo de **EMAIL**



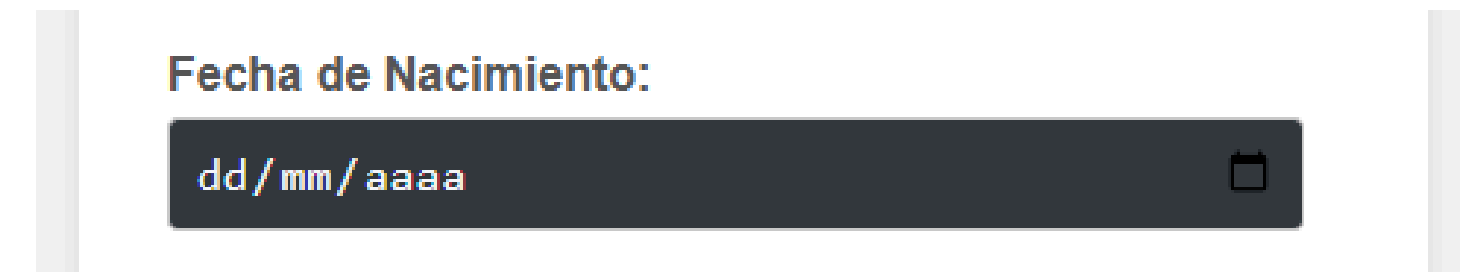
A screenshot of a web form showing an email input field. The label 'Email:' is in blue. The input field contains the text '123456' in red. The field is surrounded by a light gray border.

el formulario detectó y mostró el siguiente cartel :



A screenshot of a web form showing an email input field. The label 'Email:' is in blue. The input field contains the text '123456' in red. Below the input field, there is a white error message box with a yellow exclamation mark icon. The message reads: 'Incluye un signo "@" en la dirección de correo electrónico. La dirección "123456" no incluye el signo "@".'

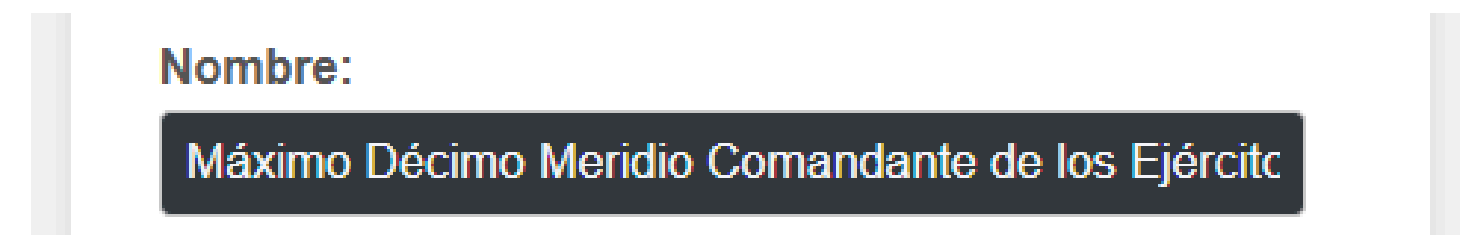
en el caso de la prueba 2 , del campo de **fecha de nacimiento** SIN completar



A screenshot of a web form showing a date input field. The label 'Fecha de Nacimiento:' is in blue. The input field contains the placeholder text 'dd/mm/aaaa' in red. The field is surrounded by a light gray border.

el formulario envió la información sin verificar que faltaba completar el campo

en el caso del campo nombre en donde se eligió un nombre demasiado largo para la prueba, se detectó que el formulario NO limitó la cantidad de nombres



A screenshot of a web form showing a name input field. The label 'Nombre:' is in blue. The input field contains the text 'Máximo Décimo Meridio Comandante de los Ejércit' in red. The field is surrounded by a light gray border.



BOOTSTRAP

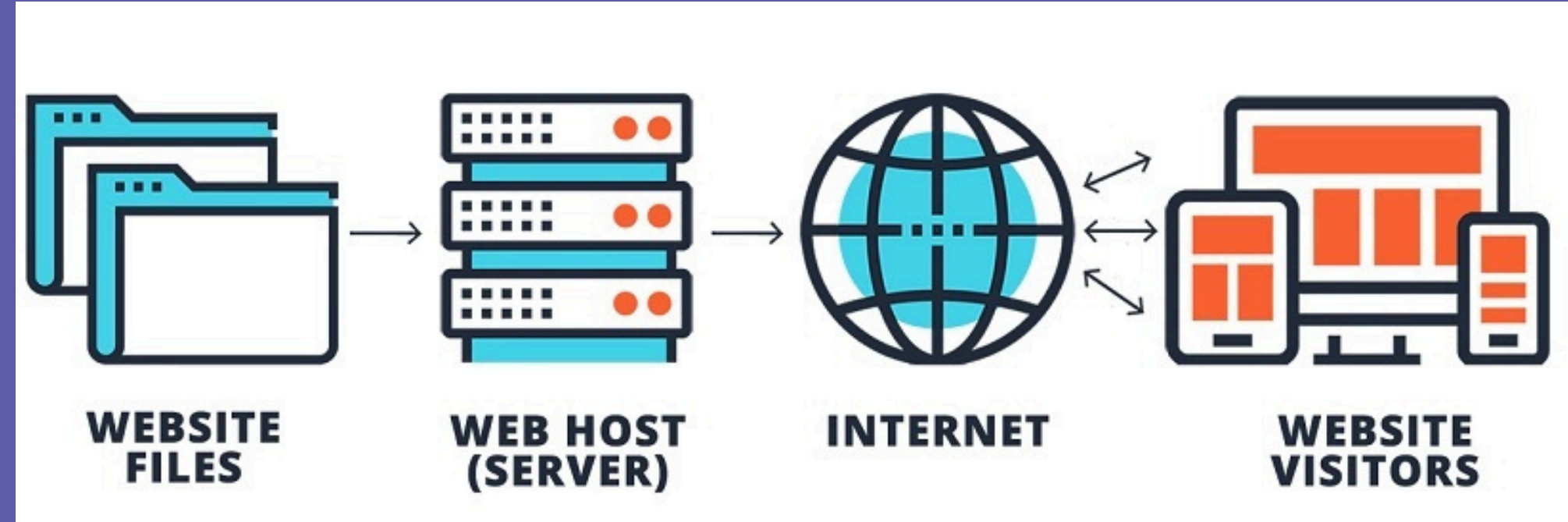
es como tener un kit de construcción súper completo con bloques prefabricados (componentes como botones, formularios, barras de navegación), herramientas estilizadas (clases CSS para diseño y tipografía) y un plano inteligente (un sistema de grillas que organiza el contenido de forma flexible).

para que sirve ?

- Desarrollo más rápido
- Diseño consistente
- Responsividad automática
- Facilidad de uso
- Gran comunidad y documentación

HOSTING

El hosting o alojamiento web es un servicio que te permite almacenar los archivos de tu sitio web en un servidor y hacerlos accesibles a través de Internet.



beneficios:

- Presencia en línea
- Funcionalidades y herramientas
- Seguridad
- Soporte técnico
- Rendimiento y velocidad

CRITERIOS

- **COSTO:** ¿CUÁNTA PLATA ESTAMOS DISPUESTOS A INVERTIR EN NUESTRO HOSTING?
- **ESCALABILIDAD:** ¿EL HOSTING PUEDE CRECER JUNTO CON NUESTRO PROYECTO?
- **RENDIMIENTO:** ¿QUÉ TAN RÁPIDO SE CARGA NUESTRO SITIO?
- **SOPORTE:** ¿QUÉ TIPO DE ASISTENCIA TÉCNICA ESTÁ DISPONIBLE? ¿HORARIOS? ¿TIEMPO DE RESPUESTA?

CONFIGURACIÓN DE HOSTING

- CREACIÓN DE UNA CUENTA EN UN SERVICIO DE HOSTING
- CONFIGURACIÓN DEL DOMINIO Y ASIGNACIÓN DE DNS
- ACCESO AL PANEL DE CONTROL DEL HOSTING
- SUBIDA DE ARCHIVOS Y CONFIGURACIÓN DE BASES DE DATOS
- CONSIDERACIONES DE SEGURIDAD Y OPTIMIZACIÓN DURANTE LA CONFIGURACIÓN

HOSTING



Buscar 2 servicios de hosting en internet

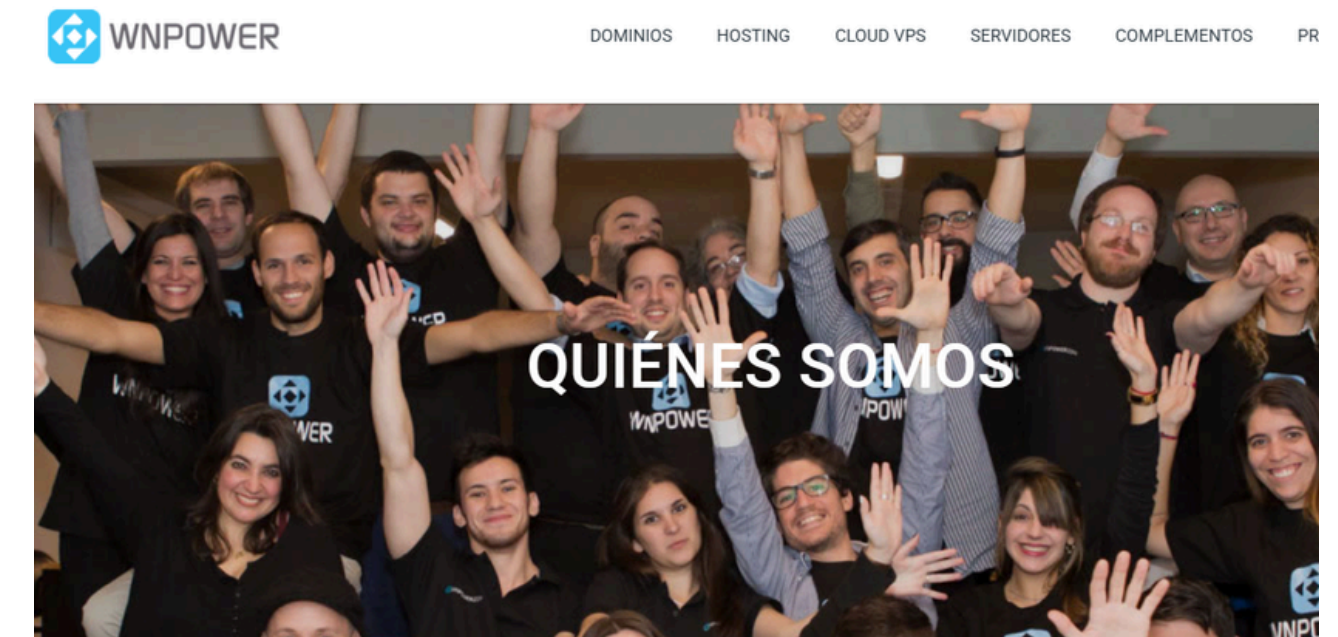
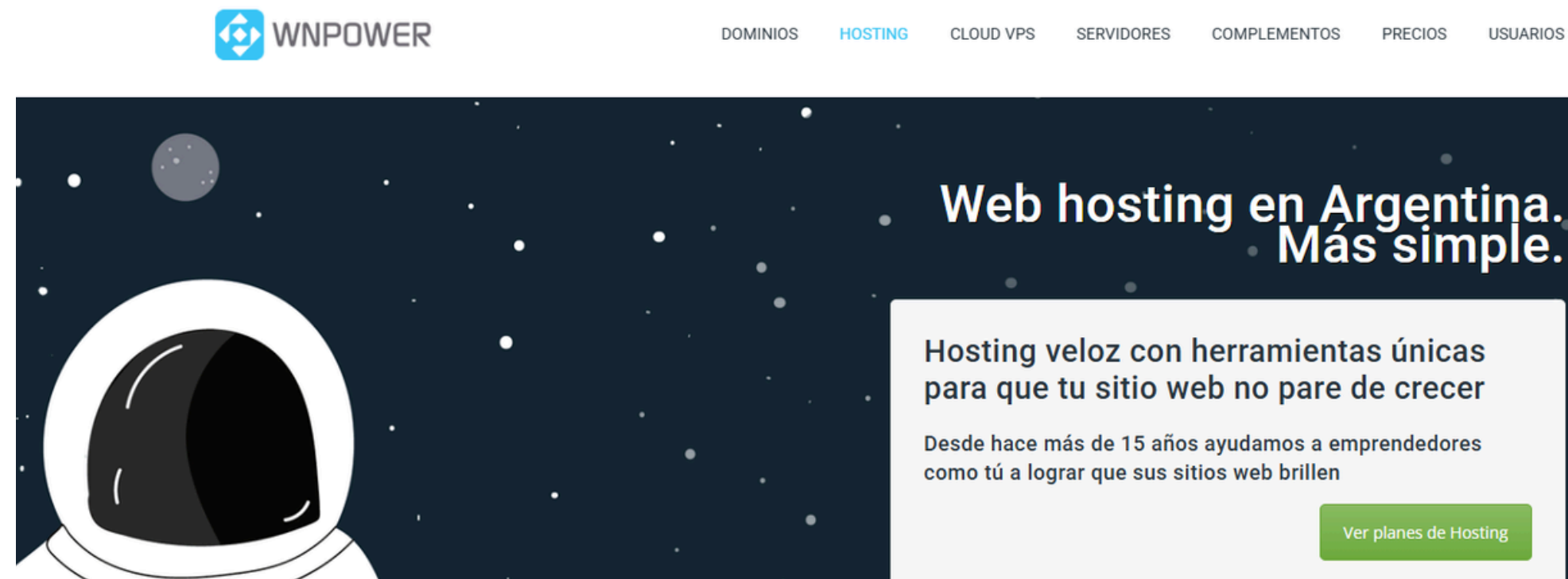
- Identificar si es gratuito y en ese caso que limitaciones tiene o si ofrece un modelo de pago por servicios.
- Si no es gratuito o corresponde a una empresa, identificar si se trata de una empresa nacional o extranjera y en qué moneda se realizan los pagos.
- Identificar los servicios ofrecidos por la empresa en función de los criterios mencionados tomando en cuenta si la contratación es mensual o anual
- Por último, realizar una comparación entre los dos servicios explorados.

servicios de hosting encontrados:

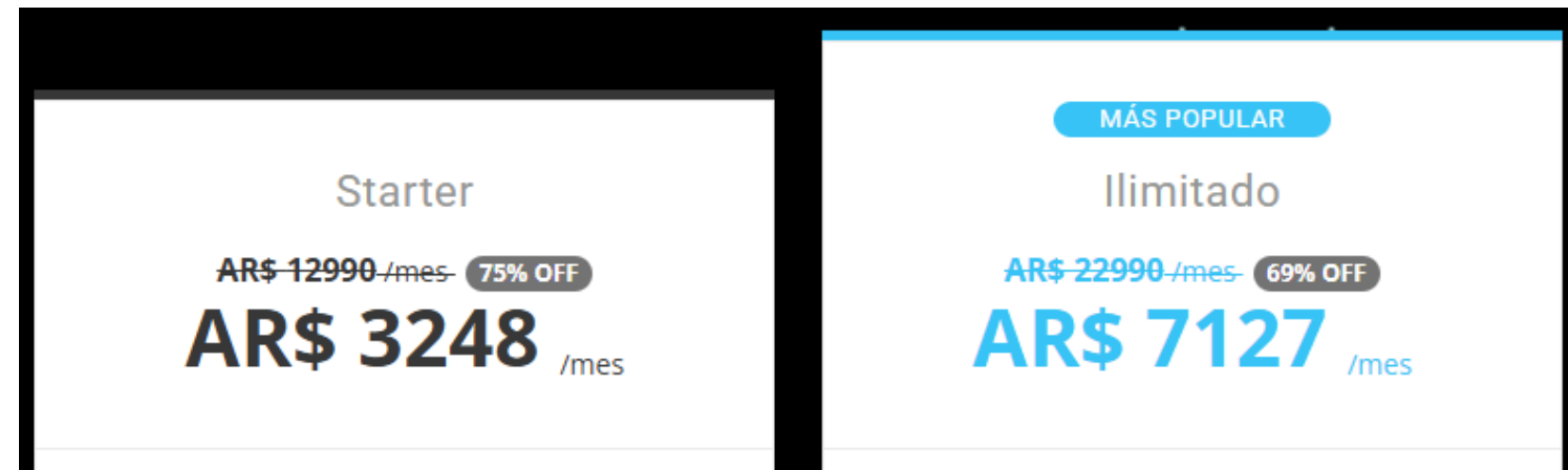
www.wnpower.com

hosgator.ar

Analisis de: www.wnpower.com



WNPOWER ofrece dos opciones de hosting pagas



plan de abono starter:

- 5GB de espacio SSD/NVMe
- Certificado SSL GRATIS!
- Cuentas de email ilimitadas
- cPanel
- PHP y MySQL
- Constructor Web
- Migrador automático

plan de abono ilimitado:

- Espacio SSD/NVMe ilimitado
- Dominio .online GRATIS! por 1 año
- Certificado SSL incluido
- Cuentas de email ilimitadas
- cPanel
- PHP y MySQL
- Constructor Web
- Migrador automático

ambas opciones de pago se abonan en moneda argentina

Analisis de: hosgator.ar

HostGator

Dominios

Hosting

Correo profesional

Contenidos

HASTA 77% OFF EN WEB HOSTING

Todo para crear tu blog

✓ Herramientas de IA para WordPress

✓ Migración de sitios web gratis

✓ Soporte en español

✓ Dominio gratis

Aprovecha la oferta

Desde \$1.889,79 ARS/mes*

Hasta 30 días para reembolso

Hasta 77% OFF

Crear con IA

Prompt personalizado

Mejorar el texto

CDN Gratuita



hosgator ofrece planes pagos mensuales ó por 3 años, con diferentes características

Recomendado			
Plan Personal	Plan Emprendedor	Plan Negocios	Plan Turbo
El primer paso de tu jornada digital	Hecho para quien está comenzando	Ideal para seguir creciendo	Perfecto para sitios web con mucho tráfico
\$302.964,00 ARS 77% OFF \$1.889,79 ARS/mes* \$68.029,20 ARS por 3 años Ahorra \$255.433,80 ARS	\$503.964,00 ARS 77% OFF \$3.149,79 ARS/mes* \$113.389,20 ARS por 3 años Ahorra \$411.673,80 ARS	\$611.964,00 ARS 77% OFF \$3.809,79 ARS/mes* \$137.149,20 ARS por 3 años Ahorra \$495.913,80 ARS	\$705.564,00 ARS 77% OFF \$4.409,79 ARS/mes* \$158.749,20 ARS por 3 años Ahorra \$567.913,80 ARS

la **principal diferencia** entre estos planes, es la capacidad de almacenamiento que va desde los 5 gb hasta 75 gb

COMPARACIÓN



WNPower se presenta con ***dos planes bien definidos***, destacándose por ofrecer alojamiento en AWS en ambos. El Plan 1 es una opción más económica con 5 GB de almacenamiento, ideal para sitios web con necesidades básicas, mientras que el Plan 2 apunta a sitios más exigentes al ofrecer espacio ilimitado y soporte prioritario, justificando su mayor precio. Ambos planes de WNPowerr incluyen una amplia gama de características como backups diarios en Amazon S3, Cloudflare en 1 clic y herramientas específicas para WordPress.



HostGator, por otro lado, ***ofrece varios planes*** que se diferencian principalmente en la ***cantidad de almacenamiento***, desde 5 GB hasta 75 GB. Si bien comparte muchas características con WNPowerr, como dominio gratis, SSL, cuentas de correo ilimitadas y backups diarios, no especifica la infraestructura de alojamiento, ni detalla la calidad del soporte en sus características principales. Su enfoque parece estar en la simplicidad y la migración fácil e ilimitada.

en resumen, creo que la elección de cualquiera de los sitios web que ofrecen hosting con distintos planes, será basada en la necesidad del cliente y en que si los requisitos de la web ameritan mayor cantidad de almacenamiento y mantenimiento